Practical 2: Object-Oriented Programming Concepts

Aim

To study and implement Object-Oriented Programming concepts such as classes, objects, inheritance, and method overriding in Python.

Theory

Object-Oriented Programming (OOP) is a programming paradigm based on the concept of objects, which bundle data (attributes) and behavior (methods) together, in contrast to procedural programming which separates data and functions. OOP rests on four main pillars: Encapsulation, which hides internal state and requires interaction through well-defined methods; Abstraction, which exposes only essential features while hiding implementation detail; Inheritance, which allows a class (child/subclass) to acquire properties and methods of another class (parent/superclass), promoting code reuse; and Polymorphism, which allows objects of different classes to be treated through a common interface. A class is a blueprint or template for creating objects, while an object is a concrete instance of a class with its own state. The __init__ method is a constructor, automatically invoked when an object is created, used to initialize instance attributes. Method overriding is a form of runtime polymorphism where a subclass provides a specific implementation of a method already defined in its parent class; when the method is called on a subclass object, Python's dynamic dispatch ensures the overridden version executes, which is demonstrated when Dog and Cat objects both respond differently to the same speak() call.

Algorithm

1. Define a base class Animal with an __init__ method and a speak() method.
2. Define subclasses Dog and Cat that inherit from Animal.
3. Override the speak() method in each subclass to return a specific sound.
4. Create a list of Animal objects containing Dog and Cat instances.
5. Loop through the list and call speak() on each object to demonstrate polymorphism.

Program

```
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        return f"{self.name} makes a sound"

class Dog(Animal):
    def speak(self):
        return f"{self.name} barks"

class Cat(Animal):
    def speak(self):
        return f"{self.name} meows"

animals = [Dog("Rex"), Cat("Whiskers")]
for a in animals:
    print(a.speak())
```

Output

```
Rex barks
Whiskers meows
```

Result / Conclusion

OOP concepts including class definition, inheritance, and method overriding were successfully implemented and verified using Python.

Viva Questions

Q1. What are the four pillars of Object-Oriented Programming?

Ans: The four pillars are: (1) Encapsulation — bundling data and methods together and restricting direct access to internal state; (2) Abstraction — exposing only relevant features and hiding complexity; (3) Inheritance — a subclass acquires properties and methods from a parent class; (4) Polymorphism — the ability of different objects to respond to the same method call in different ways.

Q2. What is the purpose of the __init__ method in Python?

Ans: __init__ is the constructor method automatically called when a new object is created from a class. It initialises the instance's attributes (self.name = name, etc.). The self parameter refers to the newly created object itself, allowing attributes to be bound to that specific instance.

Q3. What is the difference between method overriding and method overloading?

Ans: Method overriding occurs when a subclass provides its own implementation of a method that is already defined in the parent class (same name, same signature) — it enables runtime polymorphism. Method overloading means defining multiple methods with the same name but different parameter counts/types — Python does not natively support overloading; the last definition wins; it is simulated using default arguments or *args.

Q4. What is the difference between a class and an object?

Ans: A class is a blueprint or template that defines attributes and methods (e.g., class Animal). An object (instance) is a concrete entity created from that class with its own state (e.g., Dog('Rex') creates one specific instance with name='Rex'). Multiple objects can be created from the same class, each with independent attribute values.

Q5. What is polymorphism and how is it demonstrated in this practical?

Ans: Polymorphism means 'many forms' — different objects respond to the same method call differently. In this practical, both Dog and Cat inherit from Animal and override speak(). When we loop over a list of Animal references and call a.speak(), Python dynamically dispatches to the correct subclass implementation: Dog prints 'barks' and Cat prints 'meows', even though both are called through the same interface.